Discrete Math

Book subtitle

First edition

Kneeth - Note Taken By Suroj P.Student

Discrete Math

Book subtitle

First edition

Table of contents

1

Fundamentals

Key Words: *Define Algorithm and its complexity, Properties of an Algorithm, Binary Search Algorithm, Use Binary Search Algorithm, Insertion Sort Algorithm, Bubble Sort Algorithm*

1.1. Algorithm

There are many general classes of problems that arise in discrete mathematics such as:

- given a sequence of integers, find the largest one
- given a set, list all its subsets
- given a set of integers, put them in increasing order
- given a network, find the shortest path between two vertices. When presented with such a problem
- etc

Definition 1.1 (Algorithm)

An *algorithm* is a finite sequence **precise** instruction for performing a computation or for solving a problem.

♣

Note

Procedure that follows a sequence of steps that leads to the desired answer. Such a sequence of steps is called an **algorithm**

1.1.1. Algorithm for finding the maximum (largest) value in a finite sequence of integers

```
  // Find the largest Integer from given get of finite sequence.
2 Procedure max(a1,a2,a3,...,an: Integer)
3 max = a1
4 for i in 2 to n:
5   if max < ai then max = ai
6
7 return max[max is the largest integer]
```

1.2. Properties of an Algorithm.

- *Input:* An Algorithm has an input value from a specified set.
- *Output:* From each set of input values an algorithm produces output values from a specified set. The output values are the solution to the problem
- *Definitness:* The step of an Algorithm must be defined precisely.
- *Correctness:* An Algorithm should produce the correct output values for each set of input values.
- *Finiteness:* An algorithm should produce the desired output after a finite (but perhaps large) number of steps for any input in the set.
- *Effectiveness:* It must be possible to perform each step of an algorithm exactly and in a finite amount of time.
- *Generality:* The procedure should be applicable for all problems of the desired form, not just for a particular set of input values.

> **Note**
>
> To show that the algorithm is correct, we must show that when the algorithm terminates

1.3. Searching Algorithm.

- **Problem:** Locating an element in ordered list.
- **Example:** A program that checks the spelling of words searches for them in a dictionary
- **Name of this type of problem:** Searching problems.
- **General form of Searching Problems:** Locate an element x in a list of distinct elements $a_1, a_2, a_3, ..., a_n$ or determine that is not in the list.
- **Solution of Searching Problem:** The location of the term in the list that equal to x (that is, i is the solution if $x = a_i$) and 0 if the x is not in the list.

1.3.1. The Linear Search.

```
          : linear_search  :          ...   :          int
2   i := 1
3   while ( i <=n and x = ai)
4     i = i+1;
5
6   if i <= n then location := i
7   else location=0
8   return location( location of x is i if i or not found if i =0 )
```

1.3.2. Binary Search Algorithm.

- Only applicable for ordered list with **increasing** order.
- **Example:** if the terms are numbers, they are listed from smallest to largest; if they are words, they are listed in lexicographic, or alphabetic, order)
- **Algorithm:**

```
          :        search   : int        ...   :
  i := 1 ( left endpoint of the search interval)
  j := n ( right endpoint of the search interval)
  while i < j
    m :=uu floor((i+j)/2)
      if x > am then i := m + 1
      else j := m
  if x=ai then location := i
  else location := 0
  return location[location is the subscript i of the term ai
  equal to x or 0 if x not found in list]
```

1.4. Sorting Algorithm.

- **Problem:** Ordering the element of the list.
- **Example:**
  1. to produce a telephone directory it is necessary to alphabetize the names of subscribers.
- **Example:** generating a parts list requires that we order them according to increasing part number.
- **Name of the problem**: Sorting Problems.
- **Sorting:** it is the way of putting element of certain list into a another list in which the element are in increasing order.
- **Example of Sorting:**
  1. Sorting the list 7, 2, 1, 4, 5, 9 produces the list 1, 2, 4, 5, 7, 9
  2. Sorting the list d, h, c, a, f (using alphabetical order) produces the list a, c, d, f, h.

1.4.1. Bubble Sort Algorithm.

- One of the most simplest sorting Algorithm but **not** one of the most efficient Algorithm.
- Take $n!$ for n inputs

Example

Use the bubble sort to put 3, 2, 4, 1, 5 into increasing order.

◊

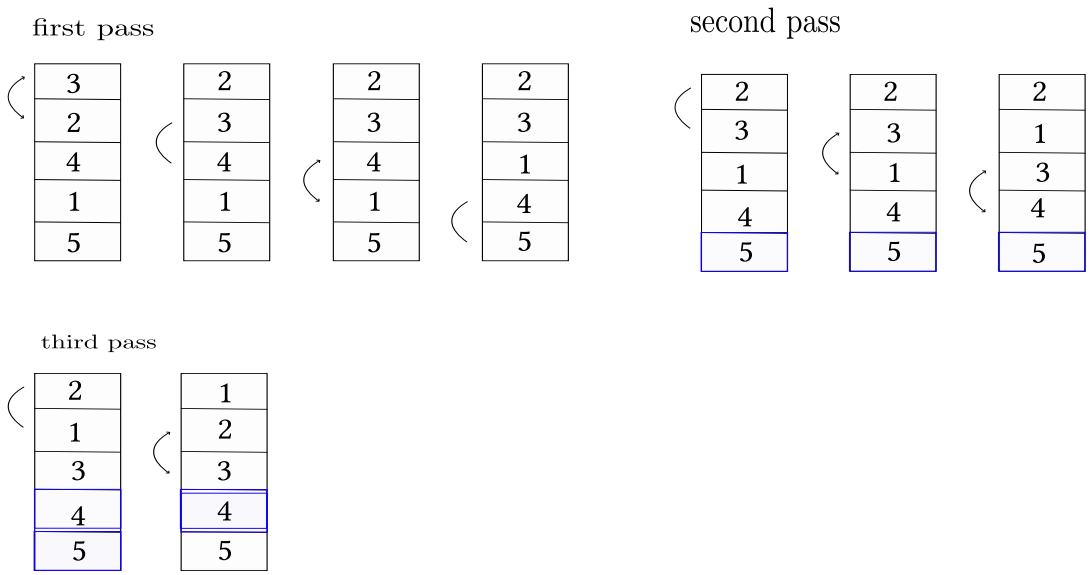


Figure 1.1 – CAPTION HERE

```
                : bubblesort       ...   :              for  >=2
2       for j = 1 to n-1
3         for i = 1 to n-2
4           if ai > a(i+1) then integerchange ai and a(i+1)
5
6       return {a1,a2,...,an} in increasing order.
7
```

1.4.1.1. In C-Programming

1. For Number.

```c
#include <stdio.h>

  int main() {
    int num[5] = {3, 2, 4, 1, 5}, temp;
    int i, j;
    for (j = 1; j <= 5 ; j++) {
      for (i = 0; i <= 3; i++) {
        if (num[i] > num[i + 1]) {
          temp = num[i];
          num[i] = num[i + 1];
          num[i + 1] = temp;
        }
      }
    }
    printf("1st Pass:\n");
    for (i = 0; i <= 4; i++) {
      printf("%d\n", num[i]);
    }
  }
```

2. For English Alphebet(or letter).

```c
#include <stdio.h>

int main(){
    char letter[]={'d','f','k','m','a','b'};
    int i,j;
    char temp;
    for(j=0;j<=5;j++){
        for(i=0;i<=4;i++){
            if(letter[i] > letter[i+1]){
                temp = letter[i];
                letter[i] = letter[i+1];
                letter[i+1] = temp;
            }
        }
    }
    for(i=0;i<=5;i++){
        printf("%c, ",letter[i]);

    }
}


output:
a, b, d, f, k, m,
```

1.4.2. Insertion Sort Algorithm.

- It is simple sorting Algorithm but usaully **not** efficient Algorithm.
- Takes $(n-1)!$ for n inputs.

```
  // Insertion Sorting Algorithm
2 Procedure: insertionSort(a1,a2,a3,...,an: Real Number) for n>=2
3 for j=2 to n
4 |  i := 1
5 |  while(aj > ai)
6 |  |  i = i+1
7 |  m := aj
8 |  for k=0 to k= j-i-1
9 |  |  a(j-k) = a(j-k-1)
10|  ai = m
11|  return (a1,a2,a3,...,an) is increasing order.
```

1.5. Greedy Algorithm.

- **Problem:** Optimizations Problem.
- Algorithms that make what seems to be the “best” choice at each step are called greedy algorithms.

2

The Growth Function.

2.1. Big-O Notation.

1. **Advantage or motivation**
   - Using this we can measure the growth of the function without worrying about the constant multiplier or smaller order term.
   - i.e Using Big-O Notation we don't have to worry about the hardware and software used to implement an Algorithm.
   - Using Big-O Notation we can assume that the different operations used in an Algorithm take the same time, which simplify the analysis considerably.
2. Uses
   - Exteremly useful to estimate the number of operations an algorithm uses as its input grows.
   - With the help of this Notation, We can determine whether it is practical to use particular algorithm to solve problem as the size of the input increase.
   - Futhermore, with the help of Big-O Notation we can compare two algorithm to determine the which is more efficient as the size of input grows.

Definition 2.1 (Big-O Notation)

Let f and g be the function from $\mathbb{N}$ or $\mathbb{R}$ to $\mathbb{R}$. We say that $f(x)$ is $O(g(x))$ if there are constant C and k such that

$$|f(x)| \leq C|g(x)| \tag{2.1}$$

whenever $x \geq k$[1].
This is read as "$f(x)$ is big-oh of $g(x)$ "

[1]Just for all $x \geq k$

Remark

Intuitively, the definition that $f(x)$ is $O(g(x))$ say what $f(x)$ grows slower than some fixed multiple of $g(x)$ as x grows without bound.

Note

The pair constant C and k in the definition are called the **witness** to the relationship $f(x)$ is $O(g(x))$.

Example 2.2

Show that $f(x) = x^2 + 2x + 1$ is $O(x^2)$.

◊

Proof::

1. WTS that there exits some constant C and k such that for all $x > k$,

$$|x^2 + 2x + 1| \leq C|x^2| \tag{2.2}$$

2. Choose $C = 3$ and $k = 2$ then we have

$$|x^2 + 2x + 1| \leq |x^2| + |2x| + |1| \tag{2.3a}$$
$$\leq |x^2| + |x^2| + 1 \tag{2.3b}$$
$$\leq 2|x^2| + |x^2| \tag{2.3c}$$
$$\leq 3|x^2| \tag{2.3d}$$

   whenever $x > 2$.

3. This proves the given statement. QED

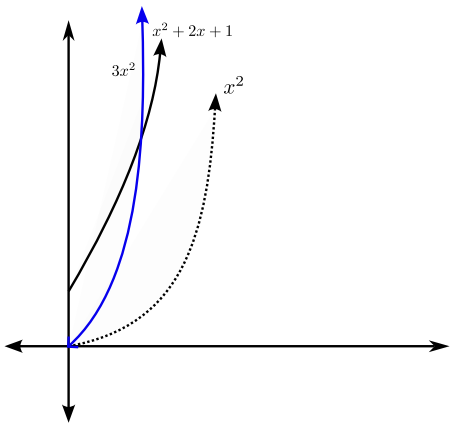


Figure 2.1 –

Remark

x^2 is also $O(x^2 + 2x + 1)$ as we have **witness** $C = 1$ and $k = 1$.

So we coin a term for this type of pair of function. We say that two function $f(x)$ and $g(x)$ that satisfy both of there big-O relationship are of the *same order*.

Note

If $f(x)$ is $O(g(x))$ then we sometimes write $f(x) = O(g(x))$

Example 2.3 (Non-Example of Big-O)

Show that n^2 is not $O(n)$.

Proof::

1. BWOC, assume that n^2 is $O(n)$.
2. Then, there must exists constant C and k such that for all $n > k$,
$$|n^2| < C|n| \tag{2.4}$$
3. Observer that $n > 0$ then
$$n^2 \leq Cn \tag{2.5a}$$
$$n \leq C \tag{2.5b}$$
4. Notice that (3) says that n is bounded by C but n is unbounded $\Longrightarrow\Longleftarrow$.

QED

2.2. Big-O Notation for some Important Function.

Theorem 2.4 ($\mathscr{P}_{n(x)}$ is $O(x^n)$)

Let $f(x) = a_n x^n + a_{n-1}x^{n-1} + ... + a_1 x + a_0$ where $a_0, a_1, ...a_n \in \mathbb{R}$. Then $f(x)$ is $O(x^n)$.

Proof:

1. From Triangle Inequalities, if $x > 1$
$$|f(x)| = |a_n x^n + a_{n-1}x^{n-1} + ... + a_1 x + a_0| \tag{2.6a}$$
$$\leq |a_n x^n| + |a_{n-1}x^{n-1}| + ... + |a_1 x| + |a_0| \tag{2.6b}$$
$$\leq |x^n|\left(|a_n| + \left|\frac{a_{n-1}}{x}\right| + ... + \left|\frac{a_1}{x^{n-1}}\right| + \left|\frac{a_0}{x^n}\right|\right) \tag{2.6c}$$
$$\leq |x^n|(|a_n| + |a_{n-1}| + ... + |a_1| + |x_0|) \tag{2.6d}$$
2. This show that
$$|f(x)| \leq C|x^n| \tag{2.7}$$
where $C = |a_n| + |a_{n-1}| + ... + |a_1| + |x_0|$ and $k = 1$.

QED

Example 2.5

How can big-O Notation be used to estimate the sum of the first n positive integer?

◊

Proof:

1. Observer that

$$\begin{aligned} \text{sum(n)} &= 1 + 2 + 3 + ... + n && \text{(2.8a)} \\ &\leq n + n + n + ... + n && \text{(2.8b)} \\ &= n^2 && \text{(2.8c)} \end{aligned}$$

2. Taking witness $C = 1$ and $k = 1$ we showed that $\sum(n)$ is $O(n^2)$.

QED

Example 2.6

Give big-O estimates for the factorial function and the log of the factorial function, where the factorial function $f(n) = n!$ defined by

$$n! = n(n-1)(n-2)\cdots 3 \cdot 2 \cdot 1 \tag{2.9}$$

◊

Proof:

1. Observe that

$$\begin{aligned} n! &= n(n-1)(n-2)\cdots 3 \cdot 2 \cdot 1 && \text{(2.10a)} \\ &\leq n \cdot n \cdot n \cdots n \cdot n && \text{(2.10b)} \\ &= n^n && \text{(2.10c)} \end{aligned}$$

2. So taking witness $C = 1$ and $k = 1$. We showed $n!$ is $O(n^n)$

3. Also observe that log is monotonic increasing function and taking log on both side makes sense in (1)

$$\begin{aligned} \log(n!) &\leq \log(n^n) && \text{(2.11a)} \\ &= n \log(n) && \text{(2.11b)} \end{aligned}$$

4. Taking "witness" "$C = 1$ and $k = 1$. We showed that $\log(n!)$ is $O(n \log n)$.

QED

3

Intro graph theory.

Definition 3.1 (two-element set)

a set $e \subseteq v$ is said to be *two-element* subsets of V iff

$$E \subseteq \{e = \{x, y\} \mid x, y \in V\}. \tag{3.1}$$

♣

Definition 3.2 (Graph)

A (simple)*Graph* G is a order pair $G = (V, E)$ consist of finite set $V = \varnothing$ and a set E of two-element subsets of V .

♣

Definition 3.3 (Graph)

A *Graph* G is a order pair $G = (V, E)$ consist of set $V = \varnothing$ and a set E is called edges..

- Each edges has either one or two vertices associate with it, called its endpoints.
- An edges is said to connect its endpoints.

♣

Remark

- A graph with an infinite vertex set (i.e $|V| = \infty$) **or** an infinite number of edges (i.e $|E| = \infty$) is called *infinite graph.*
- A graph with a finite vertex set (i.e $|V| =$ finite) **and**[1] a finite edge set is called a *finite graph*

↳ Here we only deal with finite graph.

Remark

Terminology:

[1] these two definition are **dichotomy** meaning they are negation of each other, that's why i put this margin note in "and" to let you observe in previous we have "or" and other things too.

- The element of V are called *vertices* or *nodes*.
- An element $e = \{a, b\}$ of E is called an *edge* with *end vertices* a and b .
  - We say a and b are incident with e and that a and b are *adjecent* or *neighbour* of each other, and we write $e = ab$ or $a \overset{e}{-} b$.

3.1. Types of Graph.

Definition 3.4 (Simple Graph)

A Graph G in which each edges connects two different vertices[2] and where no two edges connects to them same pair of vertices[3] is called *Simple Graph*.

Example 3.5

Given $G = (V, E)$ where $V = \{a, b, c, d\}$ and $E = \{\{a, b\}, \{a, c\}, \{b, c\}, \{b, d\}\}$.

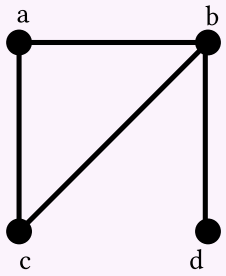


Figure 3.1 – Example of Simple Graph

Example 3.6

Given $G = (V, E)$ where $V = \{a, b, c, d\}$ and $E = \{\{a, b\}, \{a, c\}, \{b, c\}, \{b, d\}, \{d, d\}\}$.

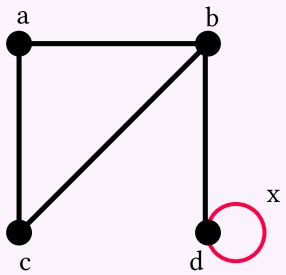


Figure 3.2 – Example of not Simple Graph

is not a simple graph as x is edges that connects same vertices.

[2] that is loops is not allowed
[3] that is not multi-graph

Example 3.7

Given Graph

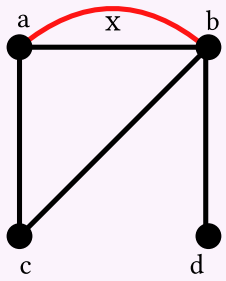


Figure 3.3 – Example of not Simple Graph

is not simple graph as there are two edges that connects same pair $\{a, b\}$ vertices. ◊

Note

In Simple Graph, when there is an edges associate to $\{u, v\}$, we can also say, without confusion, that $\{u, v\}$ is an edges of Graph.

Definition 3.8 (Multi-Graph)

A graph that **may** have **multiple edges** connecting the same pair vertices are called *multi-graph.* ♣

Example 3.9

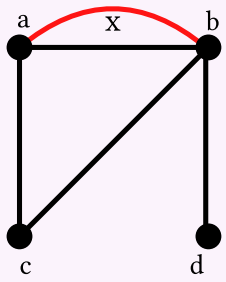


Figure 3.4 – Example of Multi-Edge Graph

◊

Note

When there are m **different edges** associated to same un-ordered pair of vertices $\{v, u\}$, we also say that $\{u, v\}$ is an edge of **multiplicity** m . For instance Previous Graph has $\{a, b\}$ of multiplicity 2.

Definition 3.10 (Loops)

In a Graph G, a loop is an edges that connects same two vertices.

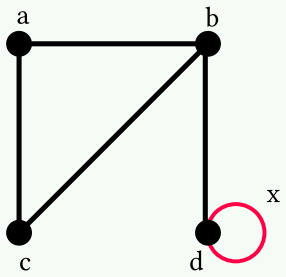


Figure 3.5 – x is loop.

♣

Definition 3.11 (Pesudo Graph)

A graph that includes **loops**, and possibly **multiple edges** connecting the same pair of vertices or a vertices to itself is called *pesudo graph.*

♣

Example 3.12

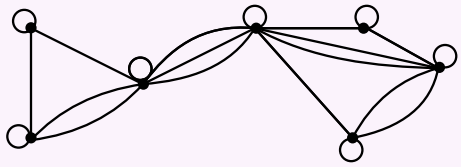


Figure 3.6 – Example of Pesudo Graph

◊

Definition 3.13 (Digraph)

A *directed graph (or digraph)* $G = (V, E)$ consists of a non-empty set of vertices V and a set of **order pair** of V (direct edges).

♣

Remark

The directed edges associated with the ordered pair (u, v) is said to be *start* at u and *end* at v .

Note

Similar definition from previous Simple Graph can be preceed

Type	Edges	Multi edge allowed	Loop allowed

Simple Graph	undirected	no	no
Multi graph	undirected	yes	no
Pseudo graph	undirected	yes	yes
Simple Digraph	directed	no	no
Directed Multi Graph	directed	yes	no
Mixed	directed and undirected	yes	yes

4

Graph Terminology and Special Type of Graph.

4.1. Basic Terminology for Undirected Graph

Definition 4.1 (Adjacency)

Let $G = (V, E)$ be a undirected graph and let $u, v \in V$. We say that u is *adjacent or neighbors* to v provided $e = \{u, v\} \in E$. The Notation $u \sim v$ means that u is adjacent to v .

- Such an edge $e = \{u, v\}$ is called *incident* with the u and v and e is said to *connect* u and v .

♣

Definition 4.2 (Nbhd of v)

The set of all neighbors(adjacent) of a vertex v of graph $G = (V, E)$, denoted by $N(v)$, is called Neighborhood of v .

$$N(v) = \{u \in V : u \text{ is adjacent of } v\} \tag{4.1}$$

♣

Remark

If $A \subseteq V$. Then the set of all vertices in G that are adjacent to at least one vertex in A is denoted by $N(A)$,

$$i.eN(A) = \bigcup_{v \in A} N(v) \tag{4.2}$$

Example 4.3

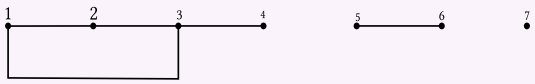
Given a graph $G = (V, E)$ where

$$V = \{1, 2, 3, 4, 5, 6\} \tag{4.3}$$

and

$$E = \{\{1,2\},\{1,2\},\{2,3\},\{3,4\},\{5,6\}\} \tag{4.4}$$

.



$N(1) = \{2,3\}$ $N(2) = \{1,3\}$ $N(3) = \{1,2,4\}$ $N(4) = \{3\}$

$N(5) = \{6\}$ $N(6) = 1$ $N(7) = \varnothing$

◊

Definition 4.4 (Degree of a Vertex)

Let $G = (V, E)$ be a **undirected** graph and let $v \in V$. The degree of v is the number of edges with which v is incident expect that a **loop** at a vertex contributes **twice** to the degree of that vertex. The degree of the vertex v is denoted by $\deg_G(v)$ or $\deg(v)$.

♣

In Example 4.3, we have

$\deg(1) = 2$ $\deg(2) = 2$ $\deg(3) = 3$ $\deg(4) = 1$

$\deg(5) = 1$ $\deg(6) = 1$ $\deg(7) = 0$

Example 4.5

Given Graph,

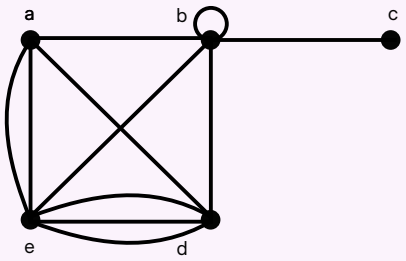


Figure 4.2 – 4.5

◊

In this Example, we have

$N(a) = \{b,d,e\}$ $N(b) = \{c,,d,e\}$ $N(c) = \{b\}$ $N(d) = \{a,b,e\}$

$$N(e) = \{a, b, d, e\}$$

and

$$\deg(a) = 4 \qquad \deg(b) = 6 \qquad \deg(c) = 1 \qquad \deg(d) = 5$$

$$\deg(e) = 6$$

Definition 4.6 (Isolated)

Let $G = (V, E)$ be a graph, a vertex of degree **zero** is called *isolated*.

In Example, 4.3, "7" is isolated.

Definition 4.7 (Pendant)

Let $G = (V, E)$ be a undirected graph, a vertex of degree **one** is called *pendant*.

4.2. The Handshaking Theorem/ Lemma

Theorem 4.8 (Handshaking theorem)

Let $G = (V, E)$ be a undirected graph with m edges. Then

$$2m = \sum_{v \in V} \deg(v) \tag{4.5}$$

proof:

1. Let $G = (V, E)$ be a undirected graph with m edges.
2. Since, each edges has two end-point, so each edges contributes 2 degree to the total sum of degree. Thus,

$$2m = \sum_{v \in V} \deg(v) \tag{4.6}$$

Q.E.D

Corollary 4.8.1

An undirected graph, has an even number of vertices of odd degree.

proof:

1. Let V_1 and V_2 be the set of vertices of even degree and the set of vertices of odd degree respectively.
2. In undirected $G = (V, E)$ with m edges, we have

$$2m = \sum_{v \in V} \deg(v) \tag{4.7a}$$

$$= \sum_{v \in V_1} \deg(v) + \sum_{v \in V_2} \deg(v) \tag{4.7b}$$

$$2m - \sum_{v \in V_1} \deg(v) = \sum_{v \in V_2} \deg(v) \tag{4.7c}$$

3. Observe that left hand side of the equation is even as $\deg(v)$ for $v \in V_1$ is even. Therefore right hand sum also must be even.
4. Since $\deg(v)$ for all $v \in V_2$ is odd and $\sum_{v \in V_2} \deg(v)$ is even.
5. Therefore, V_2 has even number of vertices as odd number of vertices is not possible.
6. Hence, There are even number of vertices of odd degree.

Q.E.D

Definition 4.9 (Adjacency)

Let $G = (V, E)$ be a digraph. We say u *is adjacent to* v and v *is adjacent from* u provided that directed edges $(u, v) \in E$.

- The vertex u is called the *initial vertex* of edge (u,v).
- The vertex v is called the *terminal vertex* of edge (u,v).

♣

Note

The initial vertex and terminal vertex of loop are same.

Definition 4.10 (Degree)

Let $G = (V, E)$ be a digraph. Then the *in degree* of vertex v is denoted by $\deg^-(v)$ is the number of edges with v as a **terminal vertex**.

The *out degree* of vertex v is denoted by $\deg^+(v)$ is the number of edges with v as a **initial vertex**.

♣

Remark

A loop at a vertex contributes 1 to both **in-degree** and **out-degree** of this vertex.

Example 4.11

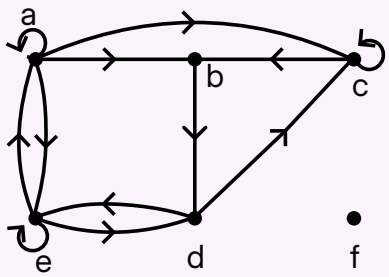


Figure 4.3 – degree for digraph

◊

We have, the in-degree in Graph are:

$$\deg^-(a) = 2 \qquad \deg^-(b) = 2 \qquad \deg^-(c) = 3 \qquad \deg^-(d) = 2$$

$$\deg^-(e) = 3 \qquad \deg^-(f) = 0$$

And the out-degree in G are:

$$\deg^+(a) = 4 \qquad \deg^+(b) = 1 \qquad \deg^+(c) = 2 \qquad \deg^+(d) = 2$$

$$\deg^+(e) = 3 \qquad \deg^+(f) = 0$$

Theorem 4.12

Let $G = (V, E)$ be a graph with directed edges. Then:

$$\sum_{v \in V} \deg^-(v) = \sum_{v \in V} \deg^+(v) = |E| \tag{4.8}$$

proof:

1. Observe that each edge has an initial vertex and a terminal vertex. Thus, the sum of in-degree and out-degree of all vertices must be equal.

$$\sum_{v \in V} \deg^-(v) = \sum_{v \in V} \deg^+(v) \tag{4.9}$$

2. From (1), each edge contributes exactly 1 to the set of initial vertices in the digraph. Therefore:

$$\sum_{v \in V} \deg^+(v) = |E| \tag{4.10}$$

3. Consequently, it follows that:

$$\sum_{v \in V} \deg^-(v) = \sum_{v \in V} \deg^+(v) = |E| \tag{4.11}$$

Q.E.D

4.3. Some Special Simple Graph.

1. **Complete Graphs**: A *complete graph on n-vertices*, denoted by K_n is a simple graph that contains exactly one edges between each pair of distinct vertices.

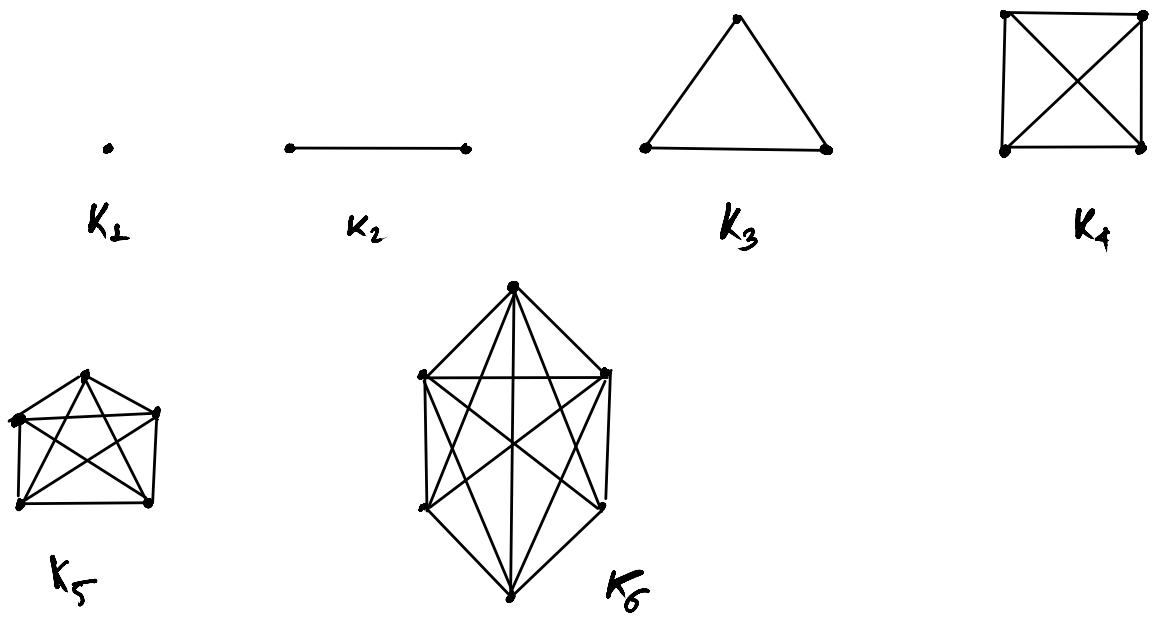


Figure 4.4 – Some Example of Complete Graphs

2. **Cycle**: A *Cycle* C_n , $n \geq 3$, consists of n-vertices $v_1, v_2, v_2, ..., v_n$ and edges $\{v_1, v_2\}, \{v_2, v_3\}, ..., \{v_{n-1}, v_n\}$

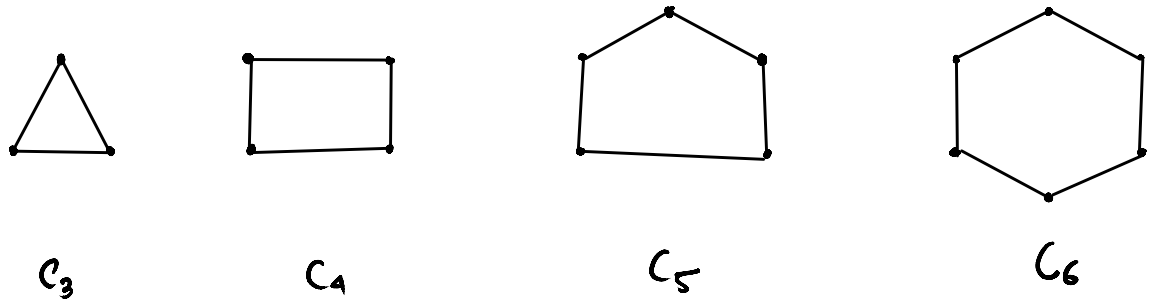


Figure 4.5 – Example of C_n

3. **Wheels**: A *wheels* W_{n+1} is graphs obtain when we **add** an aditional vertex to a cycle C_n for $n \geq 3$ and connect this new vertex to each of the n-vertices in C_n by new edges.

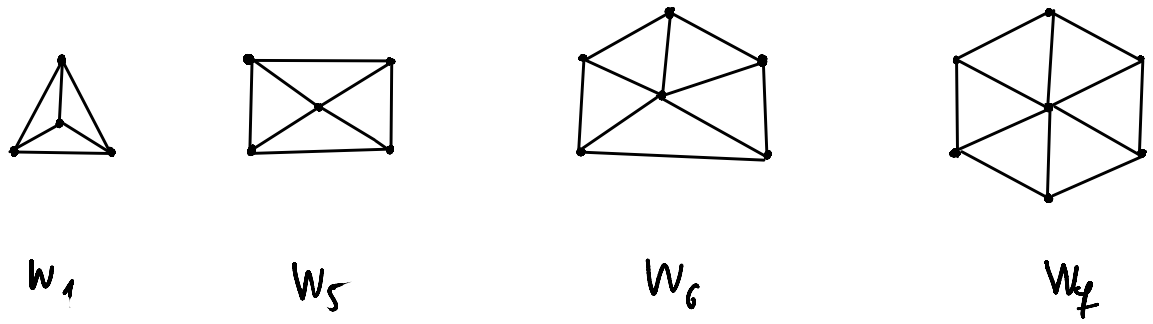


Figure 4.6 – Example of W_{n+1}

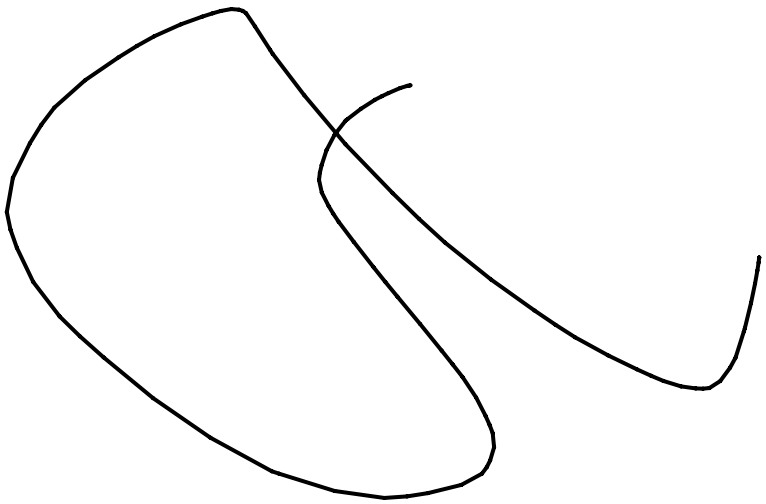


Figure 4.7 – “Rnote Generate”